COMPUTER ENGINEERING

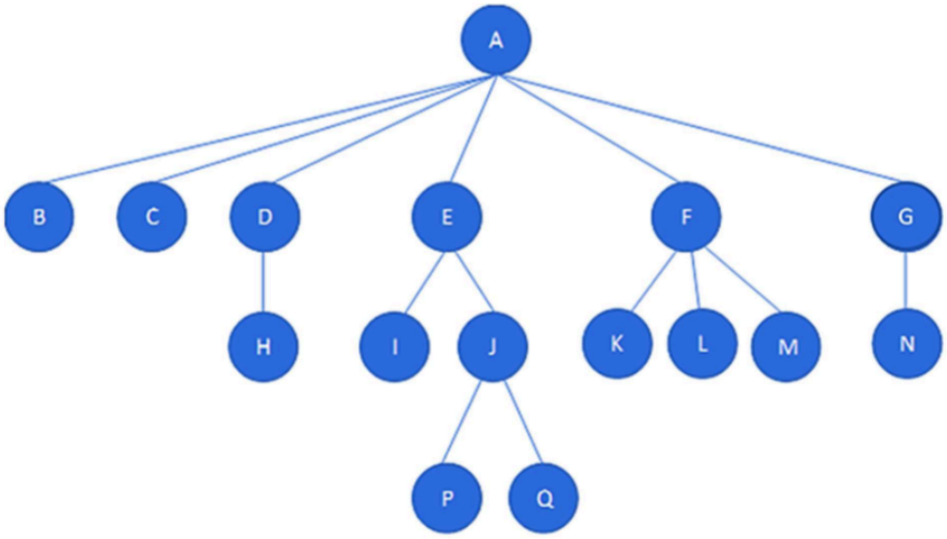
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Sep 17, 2026	**DATE SUBMITTED:**	Sep 22, 2026
NAME/S:	Campo, Ramon Brendt Canoy Hail B. Corro, Yazinel De Leon, Benj Federic C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 **Tree ADT**

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.



Tree.cpp

```
#include <iostream>
#include "Tree.h"

int main() {
    //NODE
    GeneralTree<char> A('A');
```

```
GeneralTree<char>* B = A.addChild('B');
GeneralTree<char>* C = A.addChild('C');
GeneralTree<char>* D = A.addChild('D');
GeneralTree<char>* E = A.addChild('E');
GeneralTree<char>* F = A.addChild('F');
GeneralTree<char>* G = A.addChild('G');

GeneralTree<char>* H = D->addChild('H');
GeneralTree<char>* I = E->addChild('I');
GeneralTree<char>* J = E->addChild('J');
GeneralTree<char>* P = J->addChild('P');
GeneralTree<char>* Q = J->addChild('Q');
GeneralTree<char>* K = F->addChild('K');
GeneralTree<char>* L = F->addChild('L');
GeneralTree<char>* M = F->addChild('M');
GeneralTree<char>* N = G->addChild('N');


std::cout << "\n===== Task 3.2: Traversal Output =====\n";
std::cout << "Pre-order:  "; A.preOrder();  std::cout << "\n";
std::cout << "Post-order: "; A.postOrder(); std::cout << "\n";
std::cout << "In-order:   "; A.inOrder();   std::cout << "\n";

std::cout << "\n===== Task 3.3: findData Demo =====\n";
bool found1 = false;
std::cout << "findData(1, 'J') -> ";
A.findData(1, 'J', found1);
if (!found1) std::cout << "(not found)\n";

bool found2 = false;
std::cout << "findData(2, 'Z') -> ";
A.findData(2, 'Z', found2);
if (!found2) std::cout << "(no output, key not in tree)\n";

std::cout << "\n===== Task 3.4: Add leaf 'O' as a child of G
```

```
=====\n";
    G->addChild('O');
    std::cout << "Pre-order after insert:  "; A.preOrder();
std::cout << "\n";

    bool found3 = false;
    std::cout << "findData(1, 'O') -> ";
    A.findData(1, 'O', found3);
    if (!found3) std::cout << "(not found)\n";

    return 0;
}
```

Tree.h

```
#ifndef TREE_H
#define TREE_H

#include <iostream>

template <typename T>
class GeneralTree {
private:
    T key;
    GeneralTree<T>* firstChild;
    GeneralTree<T>* nextSibling;

public:
    GeneralTree(T val) : key(val), firstChild(nullptr),
nextSibling(nullptr) {}

    ~GeneralTree() {
        delete firstChild;
        delete nextSibling;
    }
```

```
    T getKey() {
        return key;
    }

    //Creates a new node
    GeneralTree<T>* addChild(T value) {
        GeneralTree<T>* child = new GeneralTree<T>(value);

        if (firstChild == nullptr) {
            firstChild = child;
        } else {
            GeneralTree<T>* sibling = firstChild;
            while (sibling->nextSibling != nullptr) {
                sibling = sibling->nextSibling;
            }
            sibling->nextSibling = child;
        }

        return child;
    }




    void preOrder() {
        std::cout << key << " ";

        for (GeneralTree<T>* c = firstChild; c != nullptr; c =
c->nextSibling) {
            c->preOrder();
        }
    }

    void postOrder() {
        for (GeneralTree<T>* c = firstChild; c != nullptr; c =
```

```
c->nextSibling) {
            c->postOrder();
        }

        std::cout << key << " ";
    }


    void inOrder() {
        if (firstChild != nullptr) {
            firstChild->inOrder();
        }

        std::cout << key << " ";

        if (firstChild != nullptr) {
            for (GeneralTree<T>* c = firstChild->nextSibling; c !=
nullptr; c = c->nextSibling) {
                c->inOrder();
            }
        }
    }

    //Locates the node holding `value` anywhere in this subtree.
    GeneralTree<T>* find(T value) {
        if (key == value) {
            return this;
        }

        for (GeneralTree<T>* c = firstChild; c != nullptr; c =
c->nextSibling) {
            GeneralTree<T>* result = c->find(value);
            if (result != nullptr) {
                return result;
            }
        }
```

```
        return nullptr;
    }

    //Traverses using the chosen order
    void findData(int choice, T target, bool& foundFlag) {
        if (foundFlag) {
            return;
        }

        if (choice == 1) {
            if (key == target) {
                std::cout << target << " was found!" << std::endl;
                foundFlag = true;
            }
            for (GeneralTree<T>* c = firstChild; c != nullptr &&
!foundFlag; c = c->nextSibling) {
                c->findData(choice, target, foundFlag);
            }
        } else if (choice == 2) {
            for (GeneralTree<T>* c = firstChild; c != nullptr &&
!foundFlag; c = c->nextSibling) {
                c->findData(choice, target, foundFlag);
            }
            if (!foundFlag && key == target) {
                std::cout << target << " was found!" << std::endl;
                foundFlag = true;
            }
        } else if (choice == 3) {
            if (firstChild != nullptr) {
                firstChild->findData(choice, target, foundFlag);
            }
            if (!foundFlag && key == target) {
                std::cout << target << " was found!" << std::endl;
                foundFlag = true;
            }
```

```
            if (firstChild != nullptr) {
                for (GeneralTree<T>* c = firstChild->nextSibling; c
!= nullptr && !foundFlag; c = c->nextSibling) {
                    c->findData(choice, target, foundFlag);
                }
            }
        }
    }
};

#endif
```

OUTPUT

```
===== Task 3.2: Traversal Output =====
Pre-order:  A B C D H E I J P Q F K L M G N
Post-order: B C H D I P Q J E K L M F N G A
In-order:   B A C H D I E P J Q K F L M N G

===== Task 3.3: findData Demo =====
findData(1, 'J') -> J was found!
findData(2, 'Z') -> (no output, key not in tree)

===== Task 3.4: Add leaf 'O' as a child of G =====
Pre-order after insert:  A B C D H E I J P Q F K L M G N O
findData(1, 'O') -> O was found!
```

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1

F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Table 9.2

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post- order traversal

Task 3: After implementing the code for above, answer the following:

3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand. Include this as table 9-3 in section 6.

Traversal	**Result**
Pre-order	A, B, C, D, H, E, I, J, P, Q, F, K, L, M, G, N
Post-order	B, C, H, D, I, P, Q, J, E, K, L, M, F, N, G, A
In-order	B, A, C, H, D, I, E, P, J, Q, K, F, L, M, N, G

Table 9.3

3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console. Once you have the output, create and fill table 9-4 in section 6 so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in #1 and the output of your functions in #2.

```
45      void preOrder() {
46          std::cout << key << " ";
47
48          for (GeneralTree<T>* c = firstChild; c != nullptr; c = c->nextSibling) {
49              c->preOrder();
50          }
51      }
52
53      void postOrder() {
54          for (GeneralTree<T>* c = firstChild; c != nullptr; c = c->nextSibling) {
55              c->postOrder();
56          }
57
58          std::cout << key << " ";
59      }
60
61
62      void inOrder() {
63          if (firstChild != nullptr) {
64              firstChild->inOrder();
65          }
66
67          std::cout << key << " ";
68
69          if (firstChild != nullptr) {
70              for (GeneralTree<T>* c = firstChild->nextSibling; c != nullptr; c = c->nextSibling) {
71                  c->inOrder();
72              }
73          }
74      }
```

Table 9.4

3.3. Create a function called findData that will take the following parameter: “CHOICE, KEY”. Choice will determine which traversal method will be used to traverse the tree to find the second parameter KEY. If the KEY is found, the value will be displayed such that: “{KEY} was found!”. Otherwise, no output. Include your output as table 9-5 in section 6.

```
    //Locates the node holding `value` anywhere in this subtree.
    GeneralTree<T>* find(T value) {
        if (key == value) {
            return this;
        }

        for (GeneralTree<T>* c = firstChild; c != nullptr; c = c->nextSibling) {
            GeneralTree<T>* result = c->find(value);
            if (result != nullptr) {
                return result;
            }
        }

        return nullptr;
    }

    //Traverses using the chosen order
    void findData(int choice, T target, bool& foundFlag) {
        if (foundFlag) {
            return;
        }

        if (choice == 1) {
            if (key == target) {
                std::cout << target << " was found!" << std::endl;
                foundFlag = true;
            }
            for (GeneralTree<T>* c = firstChild; c != nullptr && !foundFlag; c = c->nextSibling) {
                c->findData(choice, target, foundFlag);
            }
        } else if (choice == 2) {
            for (GeneralTree<T>* c = firstChild; c != nullptr && !foundFlag; c = c->nextSibling) {
                c->findData(choice, target, foundFlag);
            }
            if (!foundFlag && key == target) {
                std::cout << target << " was found!" << std::endl;
                foundFlag = true;
            }
        } else if (choice == 3) {
            if (firstChild != nullptr) {
                firstChild->findData(choice, target, foundFlag);
            }
            if (!foundFlag && key == target) {
                std::cout << target << " was found!" << std::endl;
                foundFlag = true;
            }
            if (firstChild != nullptr) {
                for (GeneralTree<T>* c = firstChild->nextSibling; c != nullptr && !foundFlag; c = c->nextSibli
                    c->findData(choice, target, foundFlag);
                }
            }
        }
    }
};

#endif
```

Table 9.5

3.4. Add a new leaf node called which will contain the data value of “O”. Make this node the child of nodeG. Perform the findData function you created in #3, what can you observe about the output? Include your output and answers as table 9-6 in section 6

```
G->addChild('O');
std::cout << "Pre-order after insert:  "; A.preOrder(); std::cout << "\n";
```

Observation: the traversal correctly picks up the newly inserted node without any other code changes — because pre/post/in-order all walk firstChild → nextSibling pointers generically, adding a node anywhere in the tree is automatically included in every traversal the next time it runs. Its position in the output (right after G) reflects exactly where it was attached.

Table 9.6

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure’s implementation in C++

BST.cpp

```
#include <iostream>
#include "BST.h"

int main() {
    // NODE
    BinarySearchTree<int> tree(2);

    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    std::cout << "Pre-order:  "; tree.preOrder();  std::cout << "\n";
    std::cout << "In-order:   "; tree.inOrder();   std::cout << "\n";
```

```
    std::cout << "Post-order: "; tree.postOrder(); std::cout << "\n";

    return 0;
}
```

BST.h

```
#ifndef BST_H
#define BST_H

#include <iostream>

template <typename T>
class BinarySearchTree {
private:
    T key;
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

public:
    BinarySearchTree(T val) : key(val), left(nullptr), right(nullptr)
{}

    ~BinarySearchTree() {
        delete left;
        delete right;
    }

    void insert(T value) {
        if (value < key) {
            if (left == nullptr) {
                left = new BinarySearchTree<T>(value);
            } else {
                left->insert(value);
            }
        } else if (value > key) {
            if (right == nullptr) {
```

```
                right = new BinarySearchTree<T>(value);
            } else {
                right->insert(value);
            }
        }
    }

    void preOrder() {
        std::cout << key << " ";

        if (left != nullptr) {
            left->preOrder();
        }

        if (right != nullptr) {
            right->preOrder();
        }
    }

    void inOrder() {
        if (left != nullptr) {
            left->inOrder();
        }

        std::cout << key << " ";

        if (right != nullptr) {
            right->inOrder();
        }
    }

    void postOrder() {
        if (left != nullptr) {
            left->postOrder();
        }

        if (right != nullptr) {
```

```
            right->postOrder();
        }

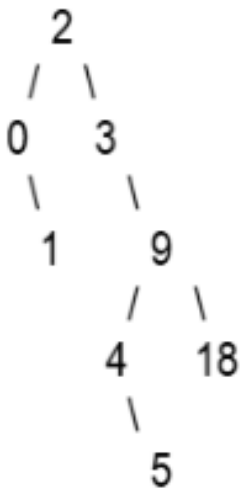
        std::cout << key << " ";
    }
};

#endif
```

OUTPUT

```
E21S1\HoA9p1\" ; if ($?) { g++ BST.cpp -o B
Pre-order:  2 0 1 3 9 4 5 18
In-order:   0 1 2 3 4 5 9 18
Post-order: 1 0 5 4 18 9 3 2
PS C:\Users\Benj Federic De Leon\Documents\
```

Structure



Traversal	Visit order
In-order	0 → 1 → 2 → 3 → 4 → 5 → 9 → 18
Pre-order	2 → 0 → 1 → 3 → 9 → 4 → 5 → 18

Post-order	1 → 0 → 5 → 4 → 18 → 9 → 3 → 2

Pre-order: 2 0 1 3 9 4 5 18
In-order: 0 1 2 3 4 5 9 18
Post-order: 1 0 5 4 18 9 3 2

Because this is a valid binary search tree, in-order traversal always yields the values in sorted ascending order, that is the defining property of a BST, and the output 0, 1, 2, 3, 4, 5, 9, 18 confirms it.
Pre-order and post-order are NOT sorted; they instead reflect the tree's shape/insertion structure (pre-order visits each subtree's root before its children, useful for copying/serializing a tree; post-order visits children before the parent, useful for safely deleting a tree bottom-up).

Code analysis

The program basically creates a Binary Search Tree with the 2 as the root node and inserts the values then smaller values will then be placed on the left and the larger values on the right. First is the insert() functions it basically finds the correct position of each value. After that it uses preOrder(), inOrder and postOrder to display the tree in different orders after that the destructor deletes the left and right nodes.

Conclusion

Summary of lessons learned:
Thus module helped us understand how tree work and how we can implement a binary search algorithm in the tree data structure. This activity in our opinion is far different yet similar from the previous activities we did, since a tree is a **non linear** data structure, it doesn't follow a path compared to linear data structures like stacks and queues, but we still need to implement a linked list in this data structure. We also learned how to implement a binary search here and the different kinds of binary searches we can implement, as well as parts of the tree(root node, key node, children node, height and dept etc.) the first searching algorithm is **In order (left< root< right)** basically it prioritizes the left child node as much as possible, if it detects there is no left node it goes to the root and then goes to the right. The 2nd searching algorithm follows a **Pre-order(root<left<right)** sequence where this time it starts from the top or the root/head node, then it goes left and when there is no more left node it goes to the right node. The last searching algorithm follows a **Post-order (Left < Right < Root)** sequence, its starts at the left most children node and goes to the right node which, if it detects there is no right node it goes to the root node. Think of this as the in order sequence but instead of going back to the node like a path, it instantly goes to the root node's children instead of going directly to the root node

I think we did well in this activity, we were able to understand most parts with everyone's cooperation and understand the concept and how the Binary Search Tree work using different methods. However we need to practice more on hard coding so we can code well without relying on too much reference and examples. Overall this activity really helped us understand the concept, properties and how Binary Search Trees work.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

COMPUTER ENGINEERING

QUEZON CITY

work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini, Grammarly

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: *Code assistance, explanation of concepts, and summarization, grammar checking*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Gemini was used to find/fix some bugs and guide us through the supplementary code, but we reviewed and understood it.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.